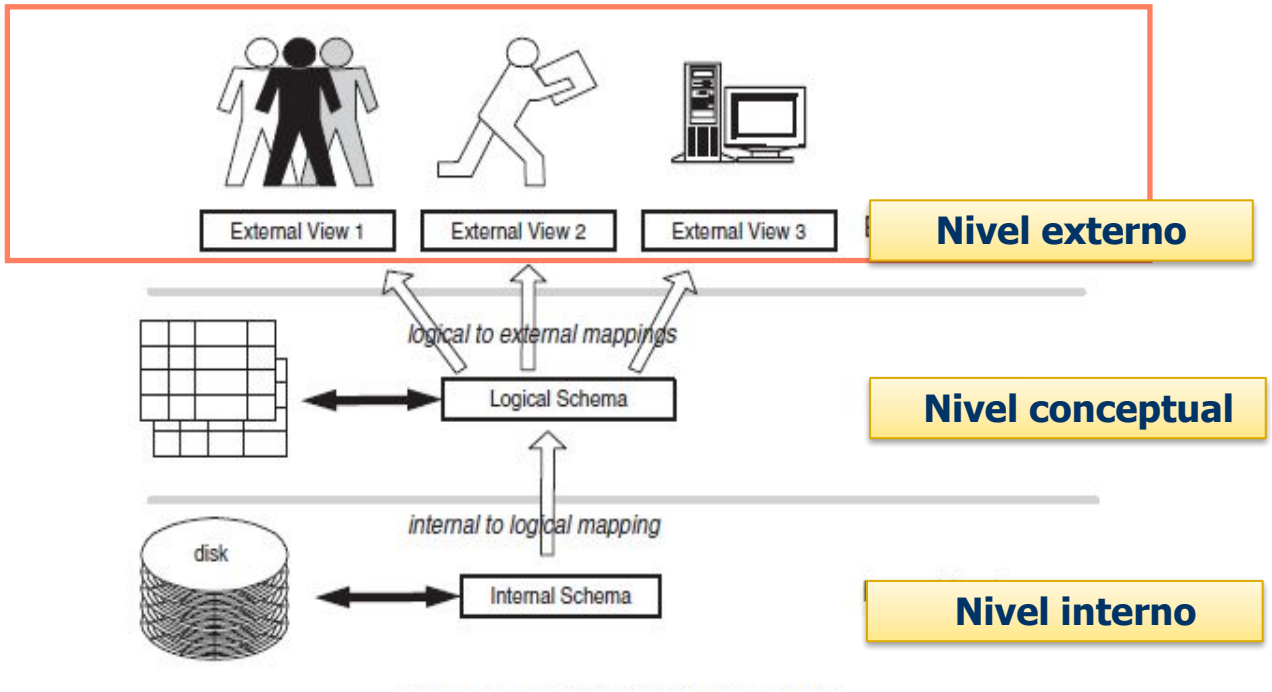
Vistas en SQL

Actualización de Vistas- Triggers Instead of

Vistas – Esquema Externo

- Las vistas forman parte del **esquema externo** de la BD
- Permiten presentar a distintos grupos de usuarios solo la información de interés, ocultando el resto de la BD (seguridad)



de: S.Sumathi, S. Esakkirajan, *Fundamentals of Relational Database Management Systems*

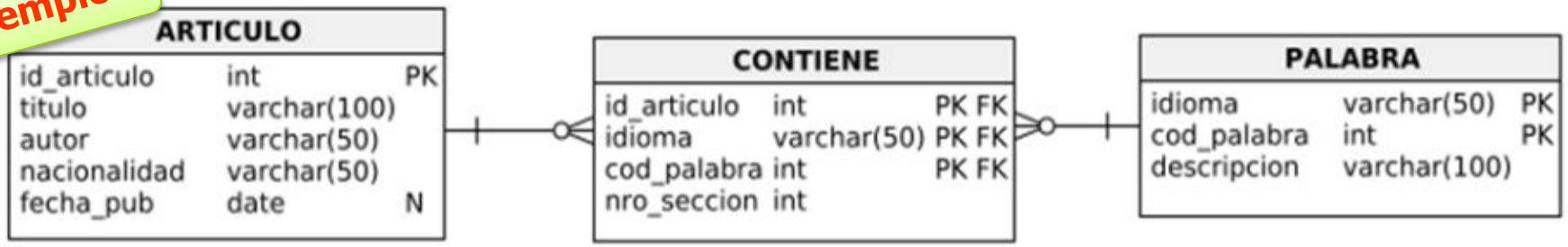
Creación de Vistas

- Una VISTA es una **tabla virtual** definida a partir de una consulta SQL sobre una o más tablas y/o vistas
- No almacena tuplas físicamente, sino que se generan al operar sobre ella (salvo si la vista está materializada)

```
CREATE VIEW nom_vista [(c1, …, cn)]
AS  expresión_consulta
[WITH [opción] CHECK OPTION];
```

- **nom_vista**: nombre de la vista (único dentro del esquema)
- **c1,…, cn**: nombres de columnas (opcionales)
  - Si no se indican, se usan los generados por la consulta
  - Se deben renombrar si resultan duplicados de la consulta
- **expresión_consulta**: consulta SQL que define la vista

Ejemplos



Vista con los datos de los artículos de nacionalidad argentina

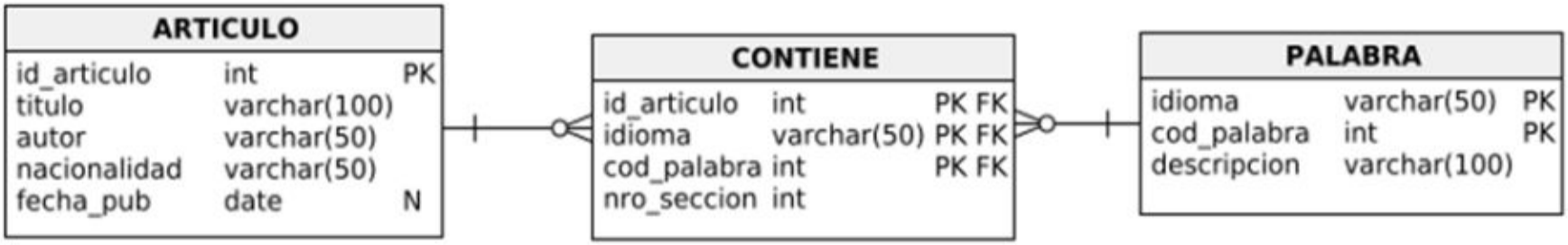
```
CREATE VIEW V_ARTICULOS_ARG AS
SELECT * FROM  ARTICULO
  WHERE nacionalidad= 'Argentina' ;
```

Vista a partir de una tabla

Vista con los datos de artículos de nacionalidad argentina publicados por autores que comienzan con M (usando la vista previa):

```
CREATE VIEW V_ARTICULOS_ARG_M AS
SELECT  * FROM V_ARTICULOS_ARG
 WHERE autor LIKE 'M%';
```

Vista a partir de otra vista



Vista con los id y título de todos los artículos de nacionalidad argentina publicados por autores que comienzan en M y la cantidad de palabras que cada uno posee

```
CREATE VIEW V_ARTICULOS_M_PALABRAS (artic, tit, cant_palabras)
AS SELECT  a.id_articulo, a.titulo, count(c.id_articulo)
FROM ARTICULO a LEFT JOIN CONTIENE c USING (id_articulo)
WHERE nacionalidad= 'Argentina' AND autor LIKE 'M%'
GROUP BY a.id_articulo, a.titulo;
```

Vista a partir de más de una tabla

```
CREATE VIEW V_ARTICULOS_M_PALABRAS_2 (artic, tit, cant_palabras)
 AS SELECT  a.id_articulo, a.titulo, count(c.id_articulo)
 FROM  V_ARTICULOS_ARG a LEFT JOIN CONTIENE c USING (id_articulo)
 GROUP BY a.id_articulo, a.titulo;
```

Vista a partir de una tabla y una vista

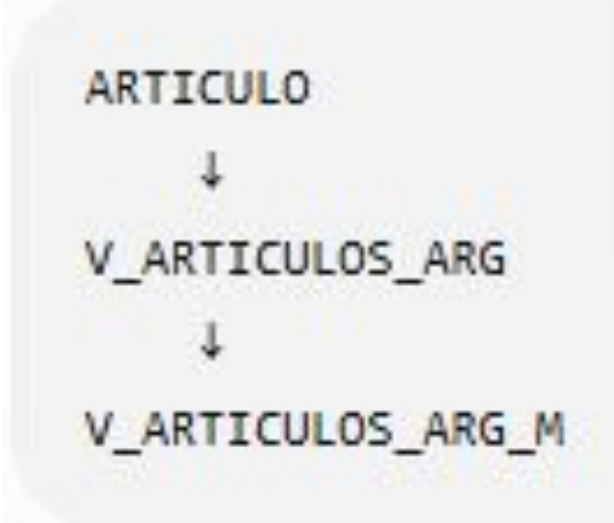
Eliminación de Vistas

Para eliminar una vista del esquema de la BD:

DROP VIEW nom_vista [opción];

opción= CASCADE: procede eliminando la definición de la vista y también los objetos dependientes de ella

opción= RESTRICT: se rechaza si hay objetos que hacen referencia a la vista (por ej. otras vistas) - *opción por defecto* (estándar SQL - PostgreSQL)



- DROP VIEW **V_ARTICULOS_ARG** CASCADE;

→ elimina V_ARTICULOS_ARG
y también V_ARTICULOS_ARG_M

- DROP VIEW **V_ARTICULOS_ARG** RESTRICT;

→ ERROR: existe una vista que depende de V_ARTICULOS_ARG

Actualizaciones Tabla Base → Vista

- Al actualizar tuplas de una tabla base → **los cambios se reflejan automáticamente en las vistas** definidas sobre ella
- El SGBD obtiene la información actualizada al acceder a las vistas, de acuerdo con su definición
- Recordar: las vistas **no mantienen copias de los datos** (salvo si son *vistas materializadas*)

Actualizaciones Vista → Tabla Base

- Una actualización en una vista **debería propagarse** a la tabla base (o vista actualizable) sobre la que se define
- **No siempre es posible la actualización automáticamente:** puede haber ambigüedades o efectos no deseados

  Ej: cómo propagar una actualización de un **campo derivado** (calculado), una **función de agregación** (COUNT, SUM, AVG, etc.)?



- El SGBD debería traducir una actualización sobre una vista en una **única operación de actualización del mismo tipo y sin ambigüedades** sobre la tabla base
- Luego -al consultar nuevamente la vista- debe reflejarse la misma actualización realizada

Vistas actualizables en SQL estándar

(a partir de una tabla/vista actualizable)

- **Según el estándar SQL:**

  Una vista definida sobre una tabla base (o vista actualizable) es automáticamente actualizable si:

  ✓ **conserva todas las columnas de la clave primaria** (necesarias para identificar unívocamente las tuplas)

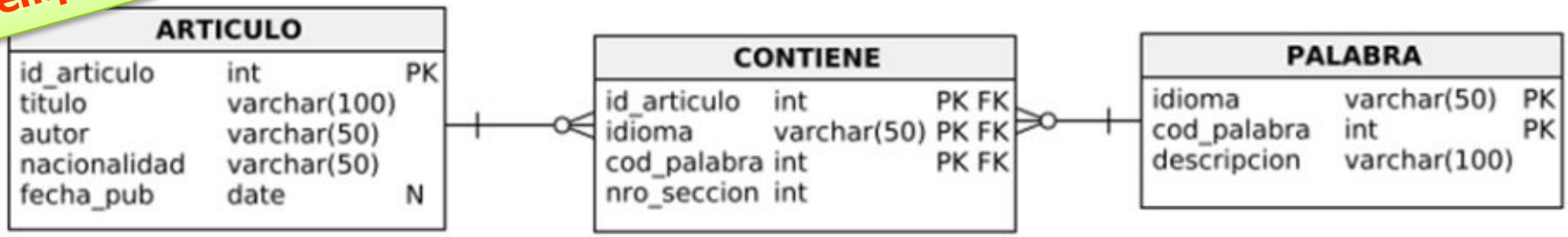
  ✓ **no contiene funciones de agregación o información derivada**

  ✓ **no incluye la cláusula DISTINCT**

  ✓ **no contiene operaciones de agrupación** (GROUP BY)

  ✓ **no incluye subconsultas en el SELECT principal** (sí se permiten en condiciones de la consulta)

Ejemplos



Estas vistas son automáticamente actualizables?

Vista con los datos de los artículos de nacionalidad argentina

```
CREATE VIEW V_ARTICULOS_ARG AS
SELECT * FROM ARTICULO
  WHERE nacionalidad= 'Argentina';
```

✓

Vista con las distintas nacionalidades de los artículos

```
CREATE VIEW V_NAC_ARTIC AS
SELECT distinct nacionalidad
FROM ARTICULO;
```

✗

Vista con los datos de los artículos con al menos una palabra en idioma español

```
CREATE VIEW V_ARTICULOS_ESP AS
SELECT * FROM ARTICULO a
WHERE EXISTS (
            SELECT 1 FROM CONTIENE c
            WHERE c.id_articulo = a.id_articulo
             AND c.idioma = 'esp' );
```

✓

Vista con el id de cada artículo y la cantidad de palabras que posee

```
CREATE VIEW V_CPAL (artic, cant_pal)
AS SELECT id_articulo, count(*)
FROM CONTIENE
 GROUP BY id_articulo;
```

✗

Migración de Tuplas

Al efectuar actualizaciones en una vista actualizable, puede ocurrir que las **tuplas afectadas dejen de pertenecer a la vista**
→ "migración" de tuplas

Ejemplo

```
CREATE VIEW  V_ARTICULOS_ARG  AS
  SELECT * FROM  ARTICULO
    WHERE nacionalidad= 'Argentina' ;
```

¿Cuál es el resultado de esta operación?

```
UPDATE V_ARTICULOS_ARG SET nacionalidad = 'España';
```

→ la modificación se propaga a la tabla ARTICULO

→ las filas afectadas dejan de satisfacer la condición de la vista

→ por lo tanto dejan de ser visibles en V_ARTICULOS_ARG (migran)

Opción de Chequeo

WITH CHECK OPTION (WCO): cláusula que evita la migración de tuplas en vistas automáticamente actualizables

```
CREATE VIEW nom_vista [(c1, …, cn)]
AS  expresión_consulta
[WITH [opción] CHECK OPTION];
```

Si se especifica WCO: una actualización sobre la vista procede si satisface la condición de la consulta, si no se rechaza la operación

- **opción= CASCADED** (*por defecto*): chequea las condiciones de la vista y de todas las vistas subyacentes (tengan o no WCO)
- **opción= LOCAL**: chequea la condición de la vista y sólo de las vistas subyacentes que también tengan WCO

Ejemplo

```
CREATE VIEW V_ARTICULOS_ARG AS
SELECT * FROM  ARTICULO
 WHERE nacionalidad= 'Argentina'
  WITH CHECK OPTION;

CREATE VIEW V_ARTICULOS_ARG_M AS
SELECT * FROM V_ARTICULOS_ARG
 WHERE autor like 'M%';

CREATE VIEW V_ARTICULOS_ARG_M_FEC AS
SELECT * FROM V_ARTICULOS_ARG_M
 WHERE fecha_pub IS NOT NULL
  WITH CASCADED / LOCAL CHECK OPTION;
```

Cuál es el resultado de:

```
INSERT INTO
V_ARTICULOS_ARG_M_FEC
(id_articulo, titulo, autor,
nacionalidad, fecha_pub)
VALUES (20, 'Nuevo
artículo', 'Juan Perez',
'Argentina', '2025-01-01');
```

si **ARTICULOS_ARG_M_FEC** tiene:

1) WCO **Cascaded** ; **2)** WCO **Local**

CASCADED → **rechaza la operación: no cumple la condición de V_ARTICULOS_ARG_M**

LOCAL → **la operación procede: no se controla la condición de V_ARTICULOS_ARG_M ya que no tiene WCO**

Vistas actualizables en SQL-estándar

(a partir de 2 o más tablas/vistas actualizables)

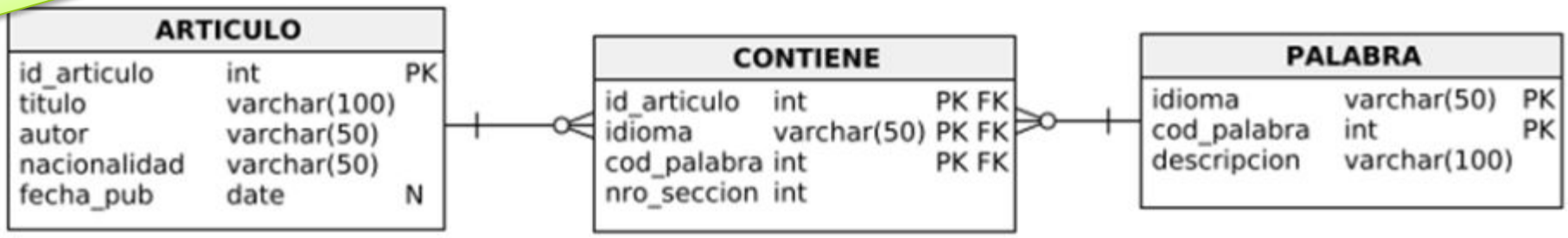
- Se obtienen mediante **ensamble** entre tablas relacionadas por **FK→PK** especificados en las **RIRs**
- No debe definirse en base a UNION, INTERSECT o EXCEPT
  (versiones posteriores a SQL-1999 sugieren incluir operaciones de intersección)

La actualización sólo puede afectar una tabla base: la que cumple la propiedad de **preservación de la clave** (la ***key preserved table*** *):

- es la tabla que tiene la misma clave de la vista
- cada tupla de esa tabla aparece a lo sumo una vez en la vista

* *según terminología de Oracle*

Ejemplo



Vista con los datos de los artículos de nacionalidad argentina y de las palabras de idioma español que contienen

```
CREATE VIEW V_ARTIC_ARG_ESP AS
    SELECT * FROM ARTICULO a JOIN CONTIENE c USING (id_articulo)
    WHERE a.nacionalidad= 'Argentina' AND c.idioma = 'Esp' ;
```

¿Cuál es la clave de la vista V_ARTIC_ARG_ESP ?

cada tupla de la vista se identifica mediante los atributos **id_articulo, idioma, cod_palabra (PK de la tabla CONTIENE)**

→ sólo se pueden hacer actualizaciones sobre los atributos provenientes de la tabla CONTIENE (tabla que propaga su clave a la vista)

¿Por qué debería propagar la actualización a CONTIENE y no a ARTICULO?

ARTICULO

id_articulo	titulo	autor	nacionalidad	...
1	tit1	Maria	España	...
2	tit2	Juan	Argentina	...
3	tit3	Ana	Argentina	...

CONTIENE

id_articulo	idioma	cod_palabra	...
2	Ing	10	...
2	Esp	20	...
2	Esp	30	...

V_ARTIC_ARG_ESP

id_articulo	titulo	autor	nacionalidad	idioma	cod_palabra	...
2	tit2	Juan	Argentina	Esp	20	...
2	tit2	Juan	Argentina	Esp	30	...

DELETE →

Ante el DELETE sobre la Vista, cuál sería el efecto de propagar el borrado a:

- **CONTIENE** → se eliminaría la tupla (2, Esp, 30, …) y la vista se regeneraría correctamente (sin incluir la tupla que se eliminó) ✓
- **ARTICULO**

  → si la RIR es cascade: se eliminaría la tupla (2, tit2, Juan, …) y también las tuplas de CONTIENE para ese artículo! ✗

  → si la RIR es restrict/no action: el SGBD rechazaría la operación (y no borraría la tupla de la vista)

Vistas actualizables en PostgreSQL

- Algunos SGBD (como SQL Server, Oracle, DB2) admiten actualizaciones en vistas con JOIN si cumplen las condiciones de preservación de clave (similar al estándar)
- pero para PostgreSQL **una vista es automáticamente actualizable si** :

✓ Tiene **una sola entrada en el FROM** (tabla o vista actualizable)

✓ El SELECT que define la vista **no incluye**:

- WITH, DISTINCT, GROUP BY, HAVING, LIMIT ni OFFSET
- operaciones de conjuntos (UNION, INTERSECT, EXCEPT)
- agregados, funciones de ventana o funciones que retornan conjuntos

✓ Puede tener una combinación de columnas

- actualizables: si corresponden a columnas actualizables de la relación subyacente
- no actualizables: si resultan de expresiones calculadas, funciones, subconsultas (de solo lectura, intentar modificarla genera error)

Explorar la documentación oficial de PostgreSQL: https://www.postgresql.org/docs/current/sql-createview.html

Vistas actualizables en PostgreSQL

Tener en cuenta: Un mismo requerimiento puede expresarse de distintas formas y quizás no todas generen una vista automáticamente actualizable

Ejemplo

Vista con los **datos de los artículos** con al menos una palabra en idioma español

```
CREATE VIEW V_ARTICULOS_ESP  AS
   SELECT a.* FROM ARTICULO a JOIN CONTIENE c USING (id_articulo)
   WHERE c.idioma = 'Esp';
```

Vista con join (no actualizable en PostgreSQL)

¿es necesario el ensamble en este caso?

```
CREATE VIEW V_ARTICULOS_ESP AS
      SELECT  *  FROM  ARTICULO a
      WHERE a.id_articulo IN (
                  SELECT c.id_articulo
                   FROM CONTIENE c WHERE c.idioma = 'Esp');
```

Vista con una subconsulta (actualizable)

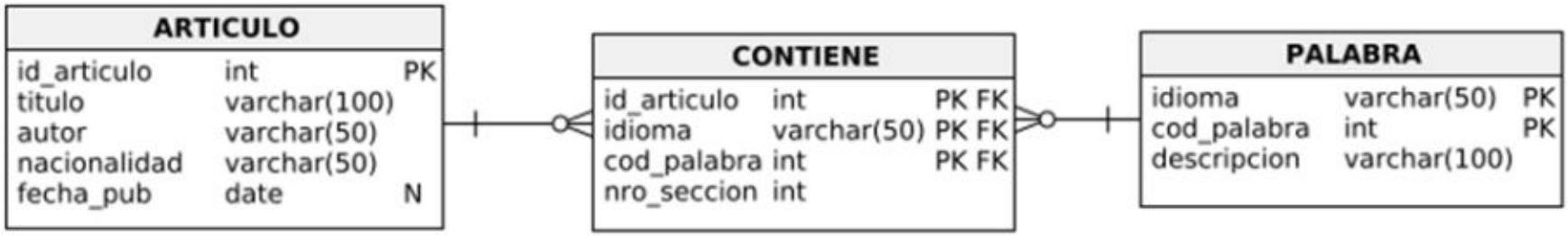
Vistas: Actualización vía triggers

- Si una vista **no es automáticamente actualizable**: se pueden definir **triggers INSTEAD OF** sobre la vista para implementar las operaciones de actualización necesarias

  → se "intercepta" la operación de actualización original y en lugar de ella, se dispara el trigger

  → es necesario implementar adecuadamente la lógica de las actualizaciones ante los distintos eventos (insert, delete, update)

- **En PostgreSQL:**
  - los triggers INSTEAD OF son FOR EACH ROW
  - NEW y OLD disponibles (igual que en triggers for each row sobre tablas)
  - No soportan condición WHEN
  - Los triggers INSTEAD OF UPDATE no admiten lista de columnas



Vista con los datos de los artículos de nacionalidad argentina y de las palabras de idioma español que contienen

```
CREATE VIEW V_ARTIC_ARG_ESP AS
  SELECT * FROM ARTICULO a
  JOIN CONTIENE c USING (id_articulo)
  WHERE a.nacionalidad= 'Argentina'
  AND c.idioma = 'Esp' ;
```

implementar la lógica para delete mediante trigger en PostgreSQL

→ delete sobre CONTIENE (*key preserved table* porque sus atributos clave identifican unívocamente cada tupla de la vista)

```
CREATE FUNCTION fn_v_artic_del()
RETURNS TRIGGER AS $$
BEGIN
  DELETE FROM CONTIENE
  WHERE id_articulo = OLD.id_articulo
   AND idioma = OLD.idioma
   AND cod_palabra = OLD.cod_palabra;
  RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_v_artic_del
INSTEAD OF DELETE ON V_ARTIC_ARG_ESP
FOR EACH ROW
EXECUTE FUNCTION fn_v_artic_del();
```

Implementación de INSERT sobre la Vista mediante trigger en PostgreSQL

```
CREATE OR REPLACE FUNCTION fn_v_artic_ins()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO CONTIENE
        (id_articulo, idioma, cod_palabra, nro_seccion)
    VALUES (NEW.id_articulo, NEW.idioma, NEW.cod_palabra,
            NEW.nro_seccion);
    RETURN NEW;
 END; $$ LANGUAGE plpgsql;

CREATE TRIGGER fn_v_artic_ins
INSTEAD OF INSERT ON V_ARTIC_ARG_ESP
FOR EACH ROW
EXECUTE FUNCTION fn_v_artic_ins();
```

Otra posible implementación en PostgreSQL para INSERT sobre la Vista

```
CREATE OR REPLACE FUNCTION fn_v_artic_ins() RETURNS TRIGGER AS $$
 BEGIN
   -- si no existe el artículo se puede insertar (están todos los datos disponibles)
   IF NOT EXISTS (
     SELECT 1 FROM ARTICULO
     WHERE id_articulo = NEW.id_articulo
   ) THEN
     INSERT INTO ARTICULO (id_articulo, titulo, autor, nacionalidad, fecha_pub)
     VALUES (NEW.id_articulo, NEW.titulo, NEW.autor, NEW.nacionalidad, NEW.fecha_pub);
   END IF;
   INSERT INTO CONTIENE (id_articulo, idioma, cod_palabra, nro_seccion)
   VALUES (NEW.id_articulo, NEW.idioma, NEW.cod_palabra, NEW.nro_seccion);
   RETURN NEW;
 END;  $$ LANGUAGE plpgsql;

 CREATE TRIGGER tr_v_artic_idioma_ins
 INSTEAD OF INSERT ON V_ARTIC_ARG_ESP
 FOR EACH ROW
 EXECUTE FUNCTION fn_v_artic_ins();
```

*¿Cómo sería la lógica para **UPDATE**?*

Vistas Materializadas

- Una vista puede ser **materializada** si se requiere alto grado de respuesta por parte de la BD (mejorar el rendimiento ante consultas frecuentes)
- Se almacena su contenido (como una tabla común) y se pueden crear índices para **mejorar la performance**
- Pero… se genera duplicación de datos → necesidad de **mantener consistencia y sincronización** con la/s tabla/s base
- Cuestiones a evaluar:

  Qué vistas deberían materializarse?

  Cómo sincronizar entre vistas materializadas y tablas base?

  - Aplicar actualización por regeneración de la vista o de forma incremental?
  - Actualización inmediata (al darse algún cambio) o periódicamente?

Vistas - consideraciones finales

Ventajas:

- Permiten mostrar diferentes datos a distintos usuarios sobre la misma BD, ocultando el resto de la información
- Facilitan la definición de consultas complejas o frecuentes
- Permiten aplicar seguridad sobre columnas (*ej: ocultar datos sensibles*) o filas (*ej. filtrar películas no aptas para menores*)
- Permiten restringir qué usuarios acceden a determinadas vistas mediante mecanismos de control de acceso

Desventajas:

- Actualizaciones restringidas: solo bajo ciertas condiciones las vistas resultan automáticamente actualizables
- Cuando una vista no es autom. actualizable, puede requerirse implementar las actualizaciones mediante triggers instead of
- La resolución de una vista puede requerir acceso a múltiples tablas, lo que puede justificar su materialización
- Las vistas materializadas implican duplicación de datos y necesidad de mantener su sincronización con las tablas base.

BIBLIOGRAFÍA

Date, C., "An Introduction to Database Systems". 8º ed., Addison Wesley, 2004

Elmasri, R., Navathe, S., "Fundamentals of Database Systems", Addison Wesley, 2011 (Cap. 5)

Ramakrishnan R., Gehrke J., "Database Management Systems", 3° ed. , McGraw-Hill, 2003 (Cap. 3 y 25)

Silberschatz, A., Korth, H, Sudarshan, S., "Database System Concepts", McGraw Hill, 2001 (Cap. 4)

Documentación PostgreSQL (link)

